Aula 12 — Matrizes

Organizando dados em linhas e colunas com listas dentro de listas

Dieimes Nunes

Retomando a Aula 11

Na aula anterior, trabalhamos com vetores e métodos de pesquisa.

Vimos:

- ordenação crescente e decrescente;
- `sort()` e `sorted()`;
- Bubble Sort;
- pesquisa sequencial;
- pesquisa binária;
- importância da ordenação.

Até agora, nossas listas eram principalmente **unidimensionais**.

Exemplo:

```
notas = [7.5, 8.0, 6.0, 9.0]
```

Hoje vamos trabalhar com dados organizados em **linhas e colunas**.

O problema da aula

Imagine as notas de três alunos em três avaliações.

	Nota 1	Nota 2	Nota 3
Ana	8	7	9
Bruno	6	8	7
Carla	9	9	10

Uma única lista pode guardar os valores.

Mas a organização por aluno e avaliação ficaria menos clara.

Precisamos representar uma **tabela**.

O que é uma matriz?

Uma matriz é uma estrutura organizada em:

- **linhas**;
- **colunas**.

Exemplo matemático:

```
8   7   9
6   8   7
9   9  10
```

Essa matriz possui:

```
3 linhas
3 colunas
```

Podemos chamá-la de matriz `3 x 3`.

Atividade 1 — Identificando linhas e colunas

Observe:

```
10   20   30   40
50   60   70   80
90  100  110  120
```

Responda no caderno:

1. Quantas linhas existem?
2. Quantas colunas existem?
3. Qual valor está na segunda linha e terceira coluna?
4. Qual valor está na terceira linha e primeira coluna?
5. Quantos elementos existem ao todo?

Depois explique como chegou às respostas.

Matrizes em Python

Python não possui uma estrutura nativa chamada “matriz” da mesma forma que algumas linguagens.

Para representar uma matriz simples, podemos usar **listas dentro de listas**.

Exemplo:

```
matriz = [
    [8, 7, 9],
    [6, 8, 7],
    [9, 9, 10]
]
```

Cada lista interna representa uma linha.

Visualizando a estrutura

```
matriz = [
    [8, 7, 9],   # linha 0
    [6, 8, 7],   # linha 1
    [9, 9, 10]   # linha 2
]
```

Dentro de cada linha, temos colunas:

```
          col 0   col 1   col 2
linha 0     8       7       9
linha 1     6       8       7
linha 2     9       9      10
```

Assim como nas listas, os índices começam em `0`.

Acessando um elemento

Para acessar um elemento usamos dois índices.

```
matriz[linha][coluna]
```

Exemplo:

```
matriz = [
    [8, 7, 9],
    [6, 8, 7],
    [9, 9, 10]
]

print(matriz[0][0])
print(matriz[1][2])
print(matriz[2][1])
```

Resultados:

```
8
7
9
```

Atividade 2 — Descobrindo posições

Considere:

```
valores = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
]
```

Sem executar, responda:

```
valores[0][2] = ?
valores[1][1] = ?
valores[2][0] = ?
valores[2][2] = ?
```

Depois escreva os comandos necessários para mostrar:

- `20`;
- `60`;
- `80`.

Só então execute para conferir.

Alterando elementos

Podemos modificar uma posição específica.

```
matriz = [
    [1, 2],
    [3, 4]
]

matriz[0][1] = 10

print(matriz)
```

Resultado:

```
[[1, 10], [3, 4]]
```

Precisamos saber exatamente qual linha e coluna serão alteradas.

Quantidade de linhas

Podemos usar `len()`.

```
matriz = [
    [1, 2, 3],
    [4, 5, 6]
]

print(len(matriz))
```

Resultado:

```
2
```

Porque existem duas listas internas, ou seja, duas linhas.

Quantidade de colunas

Se as linhas tiverem a mesma quantidade de elementos:

```
print(len(matriz[0]))
```

Resultado:

```
3
```

Isso representa a quantidade de elementos da primeira linha.

Em uma matriz regular, todas as linhas possuem o mesmo tamanho.

Atividade 3 — Criando uma matriz manualmente

Crie:

`matriz_basica.py`

Defina uma matriz `3 x 4` com números inteiros.

Depois mostre:

1. a matriz completa;
2. quantidade de linhas;
3. quantidade de colunas;
4. primeiro elemento;
5. último elemento;
6. um elemento escolhido por você.

Depois altere uma posição e mostre novamente.

Percorrendo uma matriz

Para percorrer linhas e colunas, usamos laços aninhados.

Exemplo:

```
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for linha in matriz:
    for valor in linha:
        print(valor)
```

Primeiro percorremos uma linha.

Depois percorremos os valores daquela linha.

Mostrando em formato de tabela

Podemos imprimir cada linha separadamente:

```
for linha in matriz:
    for valor in linha:
        print(valor, end="\t")
    print()
```

O parâmetro:

```
end="\t"
```

mantém os valores na mesma linha e adiciona uma tabulação.

O `print()` vazio passa para a próxima linha.

Atividade 4 — Percorrendo a matriz

Use:

```
matriz = [
    [5, 10, 15],
    [20, 25, 30],
    [35, 40, 45]
]
```

Faça o programa mostrar:

```
5   10   15
20  25   30
35  40   45
```

Depois modifique para mostrar cada elemento acompanhado de sua posição:

```
[0][0] = 5
[0][1] = 10
...
```

Percorrendo com índices

Quando precisamos conhecer linha e coluna, usamos índices.

```
for i in range(len(matriz)):
    for j in range(len(matriz[i])):
        print(f"matriz[{i}][{j}] = {matriz[i][j]}")
```

Aqui:

- `i` representa a linha;
- `j` representa a coluna.

Essa estrutura será muito usada nas atividades seguintes.

Criando uma matriz vazia

Podemos começar sem dados:

```
matriz = []
```

Depois criamos uma linha:

```
linha = []
```

Adicionamos valores à linha:

```
linha.append(10)
linha.append(20)
```

E colocamos a linha na matriz:

```
matriz.append(linha)
```

Lendo uma matriz do usuário

Exemplo `2 x 3`:

```
matriz = []

for i in range(2):
    linha = []

    for j in range(3):
        valor = int(input(f"Valor [{i}][{j}]: "))
        linha.append(valor)

    matriz.append(linha)
```

A cada repetição externa, criamos uma nova linha.

Atividade 5 — Preenchimento de matriz

Crie:

`preencher_matriz.py`

O programa deve:

1. criar uma matriz `3 x 3`;
2. solicitar os 9 valores ao usuário;
3. guardar cada valor na posição correta;
4. mostrar a matriz em formato de tabela.

Somando todos os elementos

Podemos usar um acumulador.

```
soma = 0

for linha in matriz:
    for valor in linha:
        soma = soma + valor

print(f"Soma: {soma}")
```

Observe como conhecimentos anteriores continuam sendo usados:

- matriz;
- repetição;
- acumulador.

Atividade 6 — Soma e média

A partir de uma matriz `3 x 3` preenchida pelo usuário:

1. calcule a soma de todos os elementos;
2. conte quantos elementos existem;
3. calcule a média;
4. mostre o maior valor;
5. mostre o menor valor.

Não use `max()` nem `min()` na primeira tentativa.

Pense em como adaptar a lógica aprendida com vetores.

Somando por linha

Exemplo:

```
matriz = [
    [2, 4, 6],
    [1, 3, 5],
    [10, 20, 30]
]

for i in range(len(matriz)):
    soma_linha = 0

    for j in range(len(matriz[i])):
        soma_linha += matriz[i][j]

    print(f"Soma da linha {i}: {soma_linha}")
```

O acumulador é reiniciado a cada nova linha.

Atividade 7 — Total por linha

Use uma matriz representando vendas de três vendedores durante quatro semanas.

Exemplo:

```
vendas = [
    [100, 120, 90, 150],
    [80, 110, 130, 100],
    [200, 180, 220, 210]
]
```

Mostre o total vendido por cada vendedor.

Depois identifique qual linha possui o maior total.

Somando por coluna

Agora precisamos manter a coluna fixa e percorrer as linhas.

```
for j in range(len(matriz[0])):
    soma_coluna = 0

    for i in range(len(matriz)):
        soma_coluna += matriz[i][j]

    print(f"Soma da coluna {j}: {soma_coluna}")
```

A ordem dos laços mudou.

Antes percorremos linha por linha.

Agora percorremos coluna por coluna.

Atividade 8 — Média por avaliação

Considere:

```
notas = [
    [8.0, 7.0, 9.0],
    [6.0, 8.0, 7.0],
    [9.0, 9.0, 10.0],
    [5.0, 6.5, 7.0]
]
```

Cada linha representa um aluno.

Cada coluna representa uma avaliação.

Calcule:

1. média da Avaliação 1;
2. média da Avaliação 2;
3. média da Avaliação 3.

Diagonal principal

Em uma matriz quadrada, a diagonal principal possui posições onde:

```
linha == coluna
```

Exemplo:

```
[0][0]  .      .
.     [1][1]  .
.       .    [2][2]
```

Código:

```
for i in range(len(matriz)):
    print(matriz[i][i])
```

Atividade 9 — Diagonal principal

Crie uma matriz 4 x 4.

Depois:

1. mostre todos os elementos;
2. mostre apenas a diagonal principal;
3. calcule a soma da diagonal principal;
4. conte quantos elementos da diagonal são pares.

Faça primeiro um desenho no caderno.

Matriz como tabela de lugares

Uma matriz também pode representar espaços físicos.

Exemplo de sala:

```
lugares = [
    ["L", "L", "O", "L"],
    ["O", "L", "L", "L"],
    ["L", "O", "O", "L"]
]
```

Podemos definir:

```
L = livre
O = ocupado
```

Linha e coluna identificam um lugar específico.

Atividade 10 — Mapa de lugares

Crie:

`mapa_lugares.py`

Use uma matriz com `L` e `O`.

O programa deve:

1. mostrar o mapa;
2. contar lugares livres;
3. contar lugares ocupados;
4. solicitar uma linha e coluna;
5. informar se aquele lugar está livre ou ocupado.

Valide os índices para evitar acesso fora da matriz.

Cuidado com índices inválidos

Se a matriz possui 3 linhas:

```
índices válidos: 0, 1 e 2
```

Tentar:

```
matriz[3][0]
```

causa erro.

Antes de acessar uma posição informada pelo usuário, podemos validar:

```
if 0 <= linha < len(matriz) and 0 <= coluna < len(matriz[0]):
    print(matriz[linha][coluna])
else:
    print("Posição inválida")
```

Matriz regular e linhas com tamanhos diferentes

Python permite:

```
dados = [
    [1, 2, 3],
    [4, 5],
    [6, 7, 8, 9]
]
```

Isso é possível porque são listas dentro de listas.

Porém, para representar uma matriz tradicional, normalmente queremos linhas do mesmo tamanho.

Precisamos saber qual estrutura faz sentido para o problema.

Erros comuns nesta aula

Trocar linha e coluna

```
matriz[coluna][linha]
```

quando queríamos:

```
matriz[linha][coluna]
```

Criar a linha no local errado

É importante criar:

```
linha = []
```

dentro do laço das linhas.

Esquecer um dos laços

Uma matriz normalmente precisa de dois níveis de repetição para ser percorrida por completo.

Onde estamos no curso

Agora já trabalhamos com:

```
Variável → guarda um valor
Lista    → guarda vários valores em uma dimensão
Matriz   → organiza valores em linhas e colunas
```

Mas ainda temos outro problema.

Como representar um aluno com:

- nome;
- idade;
- matrícula;
- nota;
- situação?

Na próxima aula entraremos no conceito de **registro**.

Desafio final da aula

Crie:

`sistema_notas_matriz.py`

O programa deve trabalhar com uma matriz de notas.

Regras:

1. perguntar a quantidade de alunos;
2. utilizar 3 avaliações por aluno;
3. cadastrar as notas em uma matriz;
4. mostrar a tabela completa;
5. calcular a média de cada aluno;
6. calcular a média de cada avaliação;
7. mostrar a maior nota da matriz;
8. mostrar a menor nota da matriz.

As notas devem ser validadas entre `0` e `10`.

Fechamento

Nesta aula, trabalhamos com:

- conceito de matriz;
- linhas e colunas;
- listas dentro de listas;
- acesso por dois índices;
- laços aninhados;
- preenchimento pelo usuário;
- soma por linha e por coluna;
- diagonal principal;
- tabelas e mapas;
- validação de índices.

Na próxima aula, vamos organizar dados diferentes de uma